Training a Deep Q-Network to Play Omaha Poker

Trent Houle

March 2026

Abstract

This project explores the application of Deep Q-Learning (DQN) to the game of Omaha poker, a variant of poker that presents unique challenges for reinforcement learning due to imperfect information, large state spaces, and sparse reward signals. A complete game environment was built from scratch in Python, including a hand evaluator, multiple rule-based baseline agents, and a DQN agent trained via self-play and evolutionary selection. The project analyses progressively more sophisticated training techniques, including fixed opponents, self-play, and then a population-based evolution.

1 Introduction

Poker is one of the canonical challenges in game AI. Unlike chess or Go, poker involves imperfect information, each player has private cards that their opponents cannot see. This means that a successful agent cannot simply find the objectively best move; it must reason about what opponents are likely holding and balance between exploiting weak opponents and avoiding predictable patterns.

This project focuses on Omaha poker rather than the more common Texas Hold'em. In Omaha, each player receives four hole cards and must form the best five-card hand using *exactly* two of their hole cards and *exactly* three of the five community cards. This constraint significantly expands the hand evaluation complexity compared to Texas Hold'em, where any combination of hole and community cards is valid.

The primary goal of this project is to train a DQN agent that learns to play Omaha poker through reinforcement learning, progressing from basic competence against random agents to more sophisticated play through self-play and evolutionary training.

2 Environment

2.1 Game Engine

The game environment was implemented entirely from scratch in Python, consisting of three core modules:

game.py contains the foundational data structures: `Card`, `Deck`, `Hand`, and the `Action` namedtuple. The deck supports standard 52-card representation and shuffling. Actions are represented as a named tuple with four boolean fields: `check`, `fold`, `call`, and `bet`.

agents.py contains all agent implementations, the hand evaluator, and the state vector builder. The base `Agent` class defines the interface that every agent must implement, which is a single `Action(table, agents, currBet)` method. This uniform interface allows any agent type to be swapped into a game without modifying the game loop.

main.py implements the full game loop including blind posting, dealing, and all four betting streets (preflop, flop, turn, river).

A key design decision was implementing `BettingRound` and `Round` as Python *generators* using `yield`. This allows the training loop to hook into every individual action without modifying the game logic. After each action, the generator yields a step dictionary containing the actor, action taken, current table state, and whether the hand has ended. The training loop consumes these steps to compute rewards and train the network.

2.2 Hand Evaluation

Hand evaluation in Omaha is more complex than Texas Hold'em due to the constraint that exactly two hole cards and three board cards must be used. The evaluator tries all $\binom{4}{2} \times \binom{5}{3} = 60$ combinations of hole and board cards and returns the highest scoring five-card hand.

Each five-card hand is scored as a tuple using the `EvaluateFive` function, which returns a rank from 0 (high card) to 8 (straight flush) followed by tiebreaker values. Python's native tuple comparison handles tiebreaking automatically — `(6, 14)` (full house, aces full) beats `(6, 13)` (full house, kings full) without any additional logic.

2.3 Baseline Agents

Four agent types were implemented to serve as training opponents and evaluation benchmarks:

RAgent (Random Agent) selects uniformly at random from all legal actions. When facing no bet, it chooses between checking and betting. When facing a bet, it chooses between folding, calling, and raising. This agent serves as the lower bound — any trained agent should significantly outperform it.

SAgent (Simple Agent) uses rule-based heuristics driven by estimated hand strength and pot odds. Pre-flop strength is estimated using a heuristic scoring function that rewards high card values, pairs among hole cards, suited cards, and connected cards. Post-flop strength uses the full Omaha evaluator, normalized to a $[0, 1]$ range. The agent computes pot odds (the fraction of the total pot required to call) and acts based on whether its estimated win probability exceeds the pot odds by a configurable margin.

UserAgent allows a human to play via command line input, which was used extensively during development to verify game logic correctness and observe agent behavior.

DQNAgent is described in detail in Section 3.

3 The DQN Agent

3.1 State Representation

The state vector is a 119-dimensional float32 array representing the game from the perspective of the DQN agent. The components are:

Component	Size	Encoding
Hole cards (4 cards)	20	Rank normalized to $[0, 1]$, suit one-hot
Community cards (5 cards)	25	Same as hole cards, zero-padded if undealt
Pot size	1	Normalized by total chips in play
Current bet	1	Normalized by total chips in play
Agent stack	1	Normalized by total chips in play
Average opponent stack	1	Normalized by total chips in play
Street	4	One-hot: [preflop, flop, turn, river]
Position	1	Normalized index in agent list
Hand strength	1	Best Omaha hand rank / 8.0
Action history	64	Per-opponent per-street: [fold, passive, aggressive, amount]
Total	**119**	

Several design decisions in the state representation are worth noting. Cards are encoded with separate rank and suit features rather than as a single integer index, because a single index would imply a false ordinal relationship between cards of the same rank but different suits. Chip counts are normalized against total chips currently in play rather than a fixed starting stack, making the representation robust to stack size variation throughout training.

The action history component encodes what each opponent has done across all streets, capturing information that is critical for poker strategy. A player who has bet aggressively on every street is very different from one who has been passive — this distinction is invisible without history. For each of the four streets, for each opponent, four features are stored: whether they folded, whether they acted passively (check/call), whether they acted aggressively (bet/raise), and their bet size as a fraction of the pot.

3.2 Network Architecture

The network is a fully connected feedforward network:

$$\text{Input}(119) \to \text{Linear}(256) \to \text{ReLU} \to \text{Linear}(256) \to \text{ReLU} \to \text{Linear}(256) \to \text{ReLU} \to \text{Linear}(4)$$

The output layer produces four Q-values corresponding to the four actions: fold, check, call, and bet. The agent selects the action with the highest Q-value, subject to legality constraints (e.g., checking is illegal when there is an outstanding bet).

3.3 Training Algorithm

The agent is trained using Deep Q-Learning with two standard stabilization techniques:

Experience Replay. Rather than training on each experience immediately, transitions $(s, a, r, s', \text{done})$ are stored in a replay buffer of capacity 10,000. Each training step samples a random minibatch of 64 transitions. This breaks the temporal correlation between consecutive experiences that would otherwise destabilize training.

Target Network. Two copies of the network are maintained: an online network updated at every training step, and a target network updated every 50 episodes by copying the online network weights. Q-value targets are computed using the frozen target network:

$$\text{target} = r + \gamma \cdot \max_{a'} Q_{\text{target}}(s', a') \cdot (1 - \text{done})$$

where $\gamma = 0.99$ is the discount factor. Without the target network, the agent would be chasing a moving target, leading to instability.

Epsilon-Greedy Exploration. During training, the agent selects a random action with probability ϵ and the greedy action otherwise. Epsilon is initialized to 1.0 and decayed by a factor of 0.995 each episode, with a floor of 0.05. This ensures the agent explores broadly early in training and increasingly exploits its learned policy later.

3.4 Reward Design

Reward design is one of the most critical decisions in any RL system. Poker presents a *sparse reward* problem — the agent may take many actions across four betting streets before receiving any signal about whether those actions were good.

The reward function combines a sparse terminal reward with light intermediate shaping:

Terminal reward. At the end of each hand, the agent receives a reward equal to its chip delta (chips after hand minus chips before hand), normalized by the big blind. Normalizing by the big blind keeps reward magnitudes consistent as stack sizes vary:

$$r_{\text{terminal}} = \frac{\text{chips_after} - \text{chips_before}}{\text{BIG_BLIND}}$$

Fold penalty. When the agent folds, it receives a small negative reward proportional to chips already invested, discouraging reckless folds:

$$r_{\text{fold}} = -\frac{\text{agent.In}}{\text{BIG_BLIND}}$$

This reward structure incentivizes the agent to win chips, not just to win hands — a large pot win is worth more than a small one, which is the correct objective in poker.

4 Training Methodology

4.1 Problem: Strategy Collapse

Initial experiments revealed a failure mode known as *strategy collapse*. When trained against fixed opponents (random and rule-based agents), the DQN quickly discovered that overbetting caused these agents to fold. It then converged to an aggressive overbetting strategy and stopped exploring, because this strategy was never punished by the fixed opponents. The resulting agent played poorly against any opponent that would call large bets with strong hands.

4.2 Solution: Self-Play with Snapshot Pool

To address strategy collapse, a self-play mechanism was implemented using a *snapshot pool*. Every 200 episodes, the current network weights are frozen and stored as a new opponent in the pool (up to a maximum of 10 snapshots). Each training hand, half the opponents are sampled randomly from the pool — meaning the agent plays against past versions of itself — while the other half remain rule-based agents to prevent forgetting basic strategy.

This approach forces the agent to find strategies that are robust across many opponent skill levels, since early snapshots play randomly, middle snapshots may overbet, and later snapshots likely play more balanced poker. An overbetting strategy that worked against early snapshots gets punished by later snapshots that have learned to call.

4.3 Solution: Evolutionary Training

A further refinement uses evolutionary selection across a population of agents. Rather than training a single agent, a population of agents is trained in parallel, a round-robin tournament determines the best performers, and only the top agents survive to the next generation.

Each generation proceeds as follows:

1. Train all agents for N episodes against a mixed opponent pool
2. Run a round-robin tournament (every agent vs. every other agent)
3. Rank agents by total chips won across all matchups
4. Keep the top k survivors
5. Clone each survivor into $\frac{\text{population size}}{k}$ children
6. Lower epsilon for the next generation and repeat

The epsilon schedule across generations ($1.0 \rightarrow 0.5 \rightarrow 0.25 \rightarrow 0.1 \rightarrow 0.05$) reflects a natural curriculum: early generations explore broadly to discover diverse strategies, while later generations refine the best-performing ones. Children inherit network weights from their parent but begin with a fresh replay buffer, allowing them to explore variations of their parent's strategy rather than simply replicating it.

5 Results

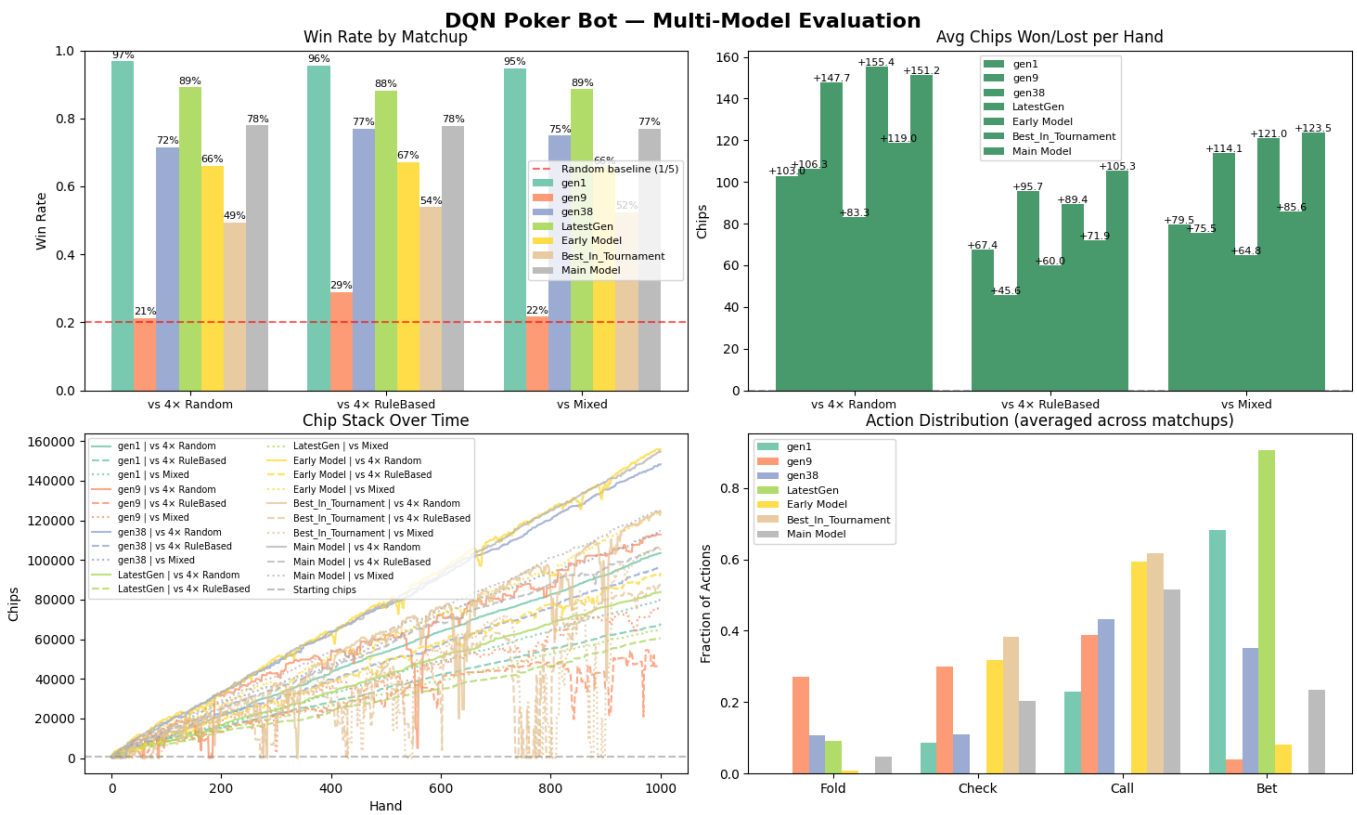

Figure 1: Evaluation of all included models, showing win rate, avg chips won/lost per hand, chip stack over time, and the action distribution.

The training curves show several expected patterns. During the early exploration phase (high epsilon), win rate jumps dramatically as the agent learns it can bet heavily and win most hands against basic opponents. As epsilon

decays and the replay buffer fills with diverse experiences, the win rate drops due to facing better opponents and then begins to climb back up. The action distribution graph reveals how the agent's strategy evolves. Early on, the agent bets aggressively (overbetting phase); after self-play snapshots begin appearing in the opponent pool, bet frequency decreases and call frequency increases, indicating the agent has learned that indiscriminate overbetting is punished.

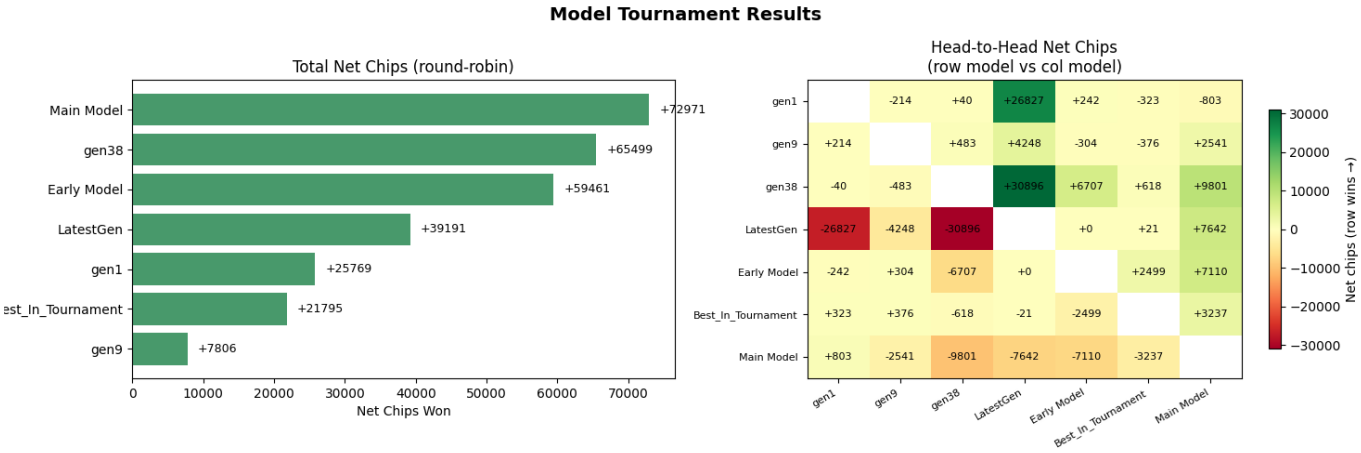

Figure 2: The results from putting each model against each other in a round robin style tournament.

The gen* models are all from the genetic algorithm that pitted models against each other, with the number in the name signifying the generation. The early model is a random model chosen from early on in the main model's training. Best_in_tournament comes from going agent by agent in the saved checkpoints and comparing them against each other to find the most successful model in training. This is to account for the possibility of an earlier model being more successful but changing in the process and being discarded.

6 How to Run

All commands should be run from the project root directory with the virtual environment activated. Each step assumes that you have completed the previous step, as some outputs are requirements for following steps (like model or statistics creation).

6.1 Training

To train the DQN agent from scratch, or resume training from a saved checkpoint:

```
python3 -m ML_Agent.train
```

The model is saved to `dqn_final.pth` upon completion, with intermediate checkpoints saved every 500 episodes to the `checkpoints/` directory. If `dqn_final.pth` already exists, training will resume automatically from the saved weights and statistics.

6.2 Evolutionary Tournament

To run the genetic algorithm, which trains a population of agents across multiple generations and selects the strongest performers:

```
python3 -m ML_Agent.tournament
```

Each generation trains all agents, runs a round-robin tournament, and clones the top survivors into the next generation with a lower exploration rate. The best agent from each generation is saved to the `evolution/` directory.

6.3 Evaluation

To evaluate the trained agent against all baseline opponents (RAgent, SAgent) and print final win rate statistics:

```
python3 -m ML_Agent.evaluate
```

This loads `dqn_final.pth` and runs the agent in inference mode (no training, no random exploration). Results are printed per opponent type.

6.4 Play Against the Bot

To play interactively against the trained agent via the command line:

```
python3 -m play
```

You will be prompted for actions (check, fold, call, bet) each time it is your turn. The trained model must exist at `dqn_final.pth` before running this command.

6.5 Dependencies

Install required packages with:

```
pip install torch numpy matplotlib
```

7 Discussion

7.1 What DQN Struggles With

DQN has a fundamental limitation in poker: it does not explicitly model opponent hands. The Q-network learns a mapping from observable state to action values, but it cannot reason counterfactually — it cannot ask "what would my opponent need to have for this call to be correct?" This is the core limitation that game-theoretic approaches like Counterfactual Regret Minimization (CFR) are designed to address.

CFR approaches poker as a mathematical optimization problem: find the strategy that minimizes the maximum regret over all possible opponent strategies. This produces a *Nash equilibrium* — a strategy that cannot be exploited regardless of what the opponent does. Real-world poker AIs like Libratus and Pluribus are built on variants of CFR rather than DQN. Implementing and comparing CFR to DQN would be a natural extension of this project.

7.2 Limitations

The primary limitation of this project is compute. Training a DQN agent to play strong poker requires significantly more than 50,000 episodes. Professional poker AI research uses billions of game iterations distributed across large compute clusters. Within the constraints of this project, the agent demonstrates clear learning over baseline agents but falls short of sophisticated strategic play.

A secondary limitation is the action space abstraction. Rather than learning continuous bet sizing, the agent's "bet" action always bets a fixed fraction of its stack. A more complete implementation would learn bet sizing as a continuous output or discretize it into multiple levels (e.g., half-pot, full-pot, all-in).

7.3 Future Work

Several extensions would meaningfully improve the agent:

- **CFR comparison**: Implement Vanilla CFR and compare its convergence and win rate against DQN, demonstrating the advantage of game-theoretic approaches in imperfect information games.
- **Continuous bet sizing**: Replace the fixed bet fraction with a learned continuous bet size output.
- **Opponent modeling**: Add an auxiliary network that predicts opponent hand strength from their action history, providing a richer signal for decision-making.
- **Deeper network**: Experiment with deeper architectures or attention mechanisms over the action history sequence.

8 Conclusion

This project demonstrates a complete pipeline for training a reinforcement learning agent to play Omaha poker, from environment design through evolutionary training. The progression from fixed opponents, to self-play, to population-based evolution each addresses a concrete failure mode of the previous approach, producing an agent that learns meaningful poker strategy within the constraints of a single-machine training budget. The architecture and training methodology are grounded in real research techniques, and the limitations identified point toward well-studied extensions in the game AI literature.

9 References & Sources

1. Deep Q-Learning (DQN) Overview
2. List of Poker Variants
3. Mnih, V. et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518, 529–533. link
4. Brown, N., & Sandholm, T. (2019). Superhuman AI for multiplayer poker. *Science*, 365(6456), 885–890. (Pluribus)link
5. Neller, T., & Lanctot, M. (2013). An introduction to counterfactual regret minimization. link
6. PyTorch DQN Tutorial